

# MVP Requirements: AI Engineering Practice Platform

---

## ("LeetCode for AI Engineering")

**Project:** B2C / LabPath pivot

**Client direction:** Yonatan Benezra

**Document version:** 1.1

**Date:** August 20, 2026

**Status:** Draft for team alignment (updated auth gate + sync)

---

## 1. Executive Summary

The product direction has changed from a **full AI learning platform (courses, LMS, marketplace)** to a **minimal practice platform**:

A small site where users land and **immediately start AI engineering exercises**, receive **AI feedback**, and have their **skill tracked**. The **first 3 questions require no login** — all progress stays in **localStorage**. From the **4th question onward**, the user **must log in**; on login, **all localStorage data syncs to the database**.

Design principles:

- Minimum text
- Minimum buttons
- Minimum navigation
- Start small; expand later
- **3 free questions**, then login to continue

This document maps the **current codebase** to the new vision and defines **what must be built, reused, removed, and deferred**.

---

## 2. Product Vision

### 2.1 What we are building

| Attribute       | Target                                                                                                   |
|-----------------|----------------------------------------------------------------------------------------------------------|
| Product type    | Question / exercise bank for <b>**AI engineering**</b>                                                   |
| Analogy         | LeetCode, but focused on ML, LLMs, RAG, agents, evals, MLOps, etc.                                       |
| Core loop       | <b>**See problem → answer (code / text / MCQ) → AI feedback → next problem**</b>                         |
| Auth            | <b>**No login for first 3 questions**</b> ; <b>**login required from question 4**</b>                    |
| UI              | Ultra-minimal; user should practice within seconds of opening the site                                   |
| Skill tracking  | <b>**localStorage for Q1–Q3**</b> ; after login, <b>**sync to database**</b> and persist on user account |
| Free tier limit | <b>**3 completed problems**</b> before login gate (configurable constant)                                |

2.2 What we are NOT building (MVP)

- AI-generated full courses
- Instructor portal / marketplace
- Subscriptions / Stripe / pricing tiers (defer)
- Admin panel (beyond internal seed scripts)
- OAuth / social login (email login only for MVP gate is acceptable)
- Multi-language i18n (English only for MVP)
- Heavy marketing site (hero, pricing, testimonials, product tour)
- SOC / network / terminal labs (unless one problem explicitly needs them)
- Learning path → "create course" upsell flows

3. Current Codebase Snapshot

3.1 Repository structure

|                  |                                                |  |
|------------------|------------------------------------------------|--|
| B2C/             |                                                |  |
| ├─ b2c-frontend/ | Next.js 16, React 19, Tailwind                 |  |
| └─ b2c-backend/  | Express, MongoDB, BullMQ, Redis, OpenRouter AI |  |

3.2 Reusable today (high value)

| Area                               | Location                                                                               | Reuse for MVP                           |
|------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------|
| Skill assessments (guest generate) | <code>`b2c-backend/.../skillAssessment.*`, `b2c-frontend/.../skill-assessment/`</code> | Public quiz flow, guest session pattern |

|                                         |                                                  |                                |
|-----------------------------------------|--------------------------------------------------|--------------------------------|
| Exercise AI grading                     | `b2c-backend/.../exercises/grading.service.ts`   | AI score + written feedback    |
| Assessment grading (MCQ / short answer) | `b2c-backend/.../assessments/grading.service.ts` | Instant + AI judge             |
| Code editor + sandbox                   | `CodeEditorLab.tsx`, `labs/code-execution/`      | Coding-style problems          |
| AI client                               | `ai-guidance/ai.client.ts`                       | Structured AI responses        |
| AI topic constants                      | `constants/aiCategories.ts`                      | Tags / categories for problems |
| Minimal public shell                    | `AssessmentSiteShell.tsx`                        | Strip-down layout              |

### 3.3 Not reusable as-is (wrong model)

| Area                                | Why                                                               |
|-------------------------------------|-------------------------------------------------------------------|
| Courses, lessons, modules           | Problems are nested inside courses; MVP needs standalone problems |
| Onboarding wizard                   | Course creation flow                                              |
| Instructor + marketplace            | Seller/catalog model                                              |
| Subscriptions + tier limits         | Paid plans not needed for MVP practice gate                       |
| Auth only at end of free trial      | Need lightweight login at Q4, not full LMS auth UX                |
| AI-generated ephemeral quizzes only | No stable <b>question bank</b> / problem catalog                  |
| 14-language i18n                    | Too much surface area for minimal UI                              |

## 4. Gap Analysis: Current vs Target

| Capability    | Current state                                        | MVP requirement                         | Gap                                  |
|---------------|------------------------------------------------------|-----------------------------------------|--------------------------------------|
| Instant start | Landing → many CTAs, assessments need topic picker   | Home = problem or auto-start session    | New default route + fewer steps      |
| Question bank | No `Problem` model; AI generates one-off assessments | Curated list of AI engineering problems | <b>New data model + seed content</b> |
| Problem types | MCQ, short answer, lesson exercises                  | MCQ + coding + prompt/design tasks      | Templates + UI per type              |
| AI feedback   | Exists for exercises; thin for MCQ                   | Rich explanation on every submission    | Prompt + UI work                     |

|                    |                                      |                                                   |                           |
|--------------------|--------------------------------------|---------------------------------------------------|---------------------------|
| No login (first 3) | Partial guest session on assessments | First 3 Q: localStorage only; Q4+: login          | Login gate + sync API     |
| Skill tracking     | User profile + course progress       | localStorage → DB after login                     | New progress + sync model |
| Minimal UI         | Full marketing + app shell + sidebar | 1–2 screens max visible                           | Remove/hide routes & nav  |
| Code run           | Sandbox execute exists               | Run + optional test cases                         | Test runner layer         |
| Navigation         | 30+ routes                           | Problem list + problem view (+ optional progress) | Route consolidation       |

## 5. MVP Functional Requirements

### 5.1 User stories (priority order)

#### P0 — Must have for launch

1. **US-01:** As a visitor, I open the site and see an exercise within **≤ 3 seconds** (no signup).
2. **US-02:** As a visitor, I can answer a problem (MCQ and/or text/code).
3. **US-03:** As a visitor, I receive **AI feedback** after submit (score or pass/fail + explanation).
4. **US-04:** As a visitor, I can complete **up to 3 problems** without logging in.
5. **US-05:** As a visitor, after each of the first 3 problems, my **performance and session data** are saved in **localStorage** (submissions, scores, topics, skill summary, completed problem slugs, guest session id).
6. **US-06:** When I try to start or access the **4th problem**, I am prompted to **log in** (or sign up) before continuing.
7. **US-07:** After login, all data from **localStorage** is **synced to the database** and linked to my user account; localStorage is cleared or marked synced.
8. **US-08:** After login, I continue from where I left off (4th problem unlocked; prior 3 attempts visible in my history).

#### P1 — Should have

9. **US-09:** As a visitor, I can browse a **short problem list** filtered by topic or difficulty (first 3 playable without login).
10. **US-10:** As a logged-in user, I see a simple **skill summary** persisted on my account.

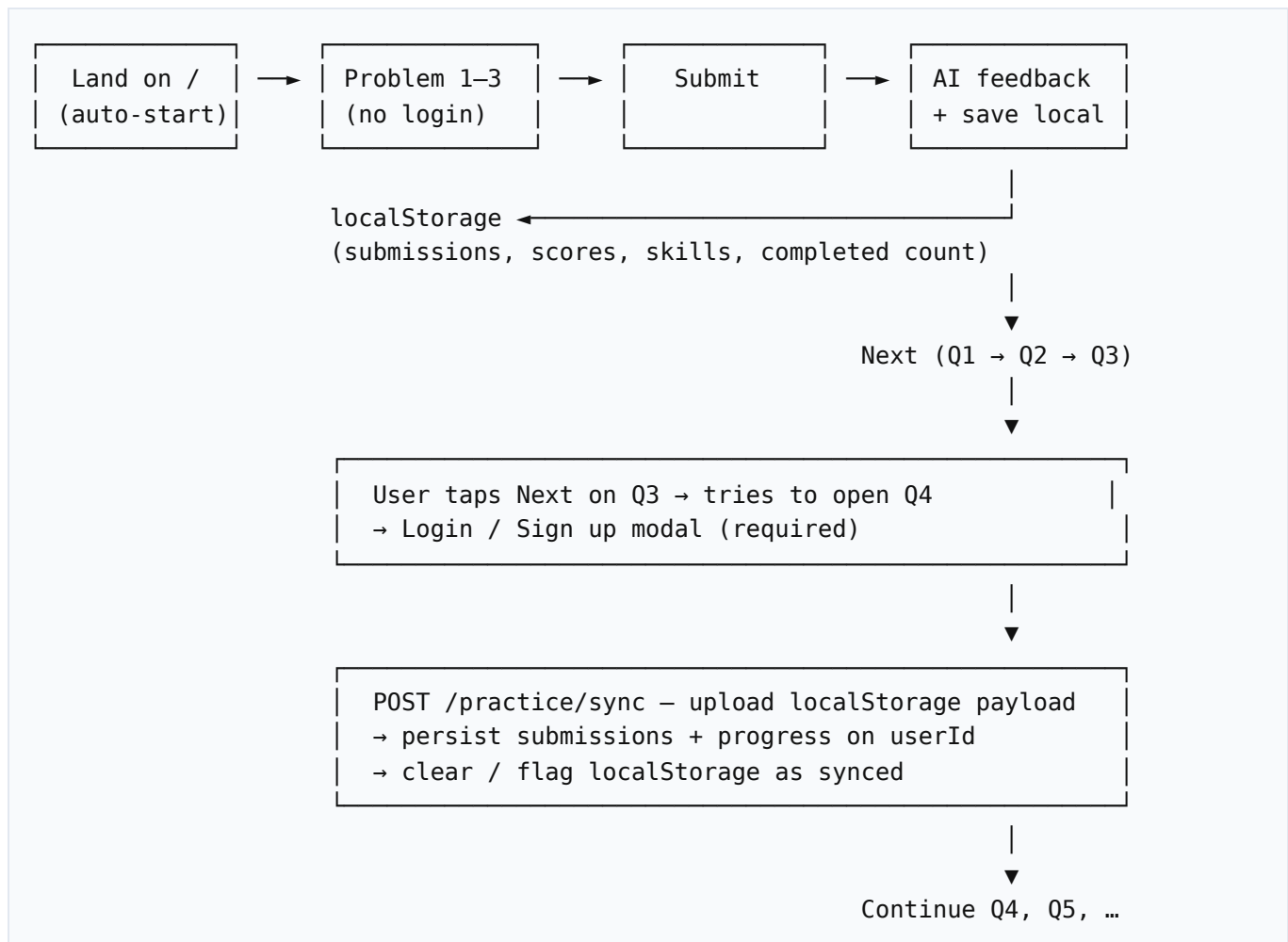
11. **US-11:** Coding problems support **Run** (sandbox) before **Submit** (AI grade).

## P2 — Nice to have (post-MVP)

12. **US-12:** Admin/internal tool to add problems without code deploy.

13. **US-13:** Signup + login merge if guest had partial progress on another device (server is source of truth after sync).

## 5.2 Core user flow (MVP)



**Login gate rule:** A problem counts toward the free limit only after **successful submit + feedback** (not merely opening the problem). Default: **FREE\_PROBLEM\_LIMIT = 3**.

### Maximum visible navigation (MVP):

- Logo / product name (optional link home)
- Problem title + **\*\*“2 / 3 free”\*\*** indicator (optional, minimal)
- Topic tag + difficulty (optional)

- Run / Submit (when applicable)
- Next (after feedback)
- Login appears **\*\*only when needed\*\*** (Q4 gate) — not in main nav during Q1–Q3

### 5.3 Problem bank requirements

#### 5.3.1 Problem entity (new)

Each problem **MUST** have:

| Field           | Required          | Notes                                                       |                  |          |                 |
|-----------------|-------------------|-------------------------------------------------------------|------------------|----------|-----------------|
| `id` / `slug`   | Yes               | URL-safe identifier                                         |                  |          |                 |
| `title`         | Yes               | Short                                                       |                  |          |                 |
| `topic`         | Yes               | From AI category list (e.g. Prompt Engineering, RAG, MLOps) |                  |          |                 |
| `difficulty`    | Yes               | `easy` \                                                    | `medium` \       | `hard`   |                 |
| `type`          | Yes               | `mcq` \                                                     | `short_answer` \ | `code` \ | `prompt_design` |
| `prompt`        | Yes               | Problem statement (markdown, keep concise)                  |                  |          |                 |
| `options`       | MCQ only          | 4 options typical                                           |                  |          |                 |
| `correctAnswer` | MCQ only          | Hidden from client when taking                              |                  |          |                 |
| `starterCode`   | Code only         | Optional template                                           |                  |          |                 |
| `rubric`        | Code / open-ended | For AI grading                                              |                  |          |                 |
| `hints`         | No (MVP)          | Defer                                                       |                  |          |                 |
| `order`         | Yes               | Default sequencing for "start immediately"                  |                  |          |                 |

**Initial content target: 15–25 seeded problems** across 5–8 AI topics (not AI-generated at runtime for MVP bank).

Suggested topic mix:

- Prompt Engineering

- LLMs & Transformers (concept MCQ)
- RAG / retrieval
- Agents & tool use
- Evaluation & metrics
- MLOps / deployment (light)
- Python for ML (code)
- Embeddings & vector search

### 5.3.2 Problem delivery

- `GET /problems` — list (topic, difficulty filters)
- `GET /problems/:slug` — single problem (no correct answer in response for MCQ)
- `GET /problems/next` — next unsolved for guest session (optional convenience endpoint)

## 5.4 Submission & AI feedback requirements

### 5.4.1 Submit

#### Questions 1–3 (guest / local only):

- Grading may run **client-side + AI API** without auth, OR via guest endpoint with rate limit
- **Do not require JWT** for submit while `completedCount < 3`
- After each submit, append result to **localStorage** (see §5.5)

#### Question 4+ (authenticated):

- `POST /problems/:slug/submit` requires **JWT**
- If user has unsynced localStorage, block Q4 until sync completes (or sync automatically immediately after login)

#### Request body (guest and authed):

```
{
  "guestSessionId": "uuid",
  "answer": "...",
  "code": "..."
}
```

### 5.4.2 Feedback response MUST include

| Field | Description |
|-------|-------------|
|-------|-------------|

|                                                |                                                      |
|------------------------------------------------|------------------------------------------------------|
| <code>`correct`</code> or <code>`score`</code> | Boolean for MCQ; 0–100 for open/code                 |
| <code>`feedback`</code>                        | AI-generated explanation (2–5 sentences, actionable) |
| <code>`correctAnswer`</code>                   | Show after submit for MCQ (optional for learning)    |
| <code>`topic`</code>                           | For skill aggregation                                |
| <code>`submissionId`</code>                    | For history                                          |

### 5.4.3 Grading rules

| Type          | MVP grading                                                                                    |
|---------------|------------------------------------------------------------------------------------------------|
| MCQ           | Rule-based instant; optional AI explanation if wrong                                           |
| Short answer  | AI judge (reuse <code>`assessments/grading.service.ts`</code> )                                |
| Code          | AI rubric grade (reuse <code>`exercises/grading.service.ts`</code> ); P1: add test-case runner |
| Prompt design | AI rubric against criteria                                                                     |

**Latency target:** Feedback visible within **≤ 15 seconds** (sync grading acceptable for MVP; async queue optional).

## 5.5 Progress, localStorage & login sync

### 5.5.1 Free tier limit

| Constant                          | Value                      | Notes                                                  |
|-----------------------------------|----------------------------|--------------------------------------------------------|
| <code>`FREE_PROBLEM_LIMIT`</code> | <b>**3**</b>               | Problems completed with feedback before login required |
| Counting rule                     | Submit + feedback received | Opening a problem does not consume a free slot         |

### 5.5.2 localStorage schema (guest, Q1–Q3)

**Key:** bina-practice-guest (single JSON object)

```
{
  "version": 1,
  "guestSessionId": "uuid",
  "freeLimit": 3,
  "completedCount": 2,
```



```

"synced": false,
"lastProblemSlug": "prompt-engineering-basics",
"submissions": [
  {
    "problemSlug": "llm-fundamentals-1",
    "topic": "Artificial Intelligence",
    "difficulty": "easy",
    "type": "mcq",
    "answer": "B",
    "score": 100,
    "correct": true,
    "feedback": "...",
    "submittedAt": "2026-08-20T12:00:00.000Z"
  }
],
"skillByTopic": {
  "Prompt Engineering": {
    "attempted": 1,
    "passed": 1,
    "avgScore": 85,
    "level": "beginner"
  }
},
"completedSlugs": ["llm-fundamentals-1", "prompt-engineering-basics"]
}

```

### Must persist locally until login:

- Each submission (answer, score, feedback, timestamp)
- Aggregated skill-by-topic metrics
- List of completed problem slugs
- `guestSessionId` (reuse existing pattern)
- `completedCount` (for gate check)

### 5.5.3 Login gate (Q4)

**Trigger:** User attempts to load the next problem when `completedCount >= FREE_PROBLEM_LIMIT` and user is **not authenticated**.

#### UX (minimal):

- Modal or inline gate: **"Sign in to continue practicing"**
- Primary: **\*\*Log in\*\*** | Secondary: **\*\*Sign up\*\***
- No full-page marketing; return to same problem after auth
- Do **\*\*not\*\*** lose Q1–Q3 data — remain in `localStorage` until sync succeeds

**Already logged in:** Skip gate; if `localStorage` has `synced: false`, run sync once on app load.

#### 5.5.4 Sync on login (localStorage → database)

**Endpoint:** POST /practice/sync (authenticated)

**Request body:** Full bina-practice-guest payload (or subset validated server-side)

**Server actions:**

1. Validate user JWT
2. Idempotent merge: upsert submissions by (userId, problemSlug, submittedAt) or clientSubmissionId
3. Recompute / merge skillByTopic on user profile
4. Mark guest submissions as claimed (guestSessionId → userId)
5. Return { synced: true, mergedCount, userProgress }

**Client actions after successful sync:**

1. Set synced: true in localStorage OR remove bina-practice-guest key
2. Redirect user to Q4 (next unsolved problem)
3. Subsequent submits go directly to DB under userId

**Failure handling:**

- If sync fails, keep localStorage intact; show retry
- Do not unlock Q4 until sync succeeds (avoid data loss)

#### 5.5.5 Tracked metrics

Per topic (local first, then server after sync):

- Problems attempted
- Problems passed (score ≥ 70% or MCQ correct)
- Average score
- Derived level: `beginner` | `intermediate` | `advanced`

#### 5.5.6 UI

- Optional minimal counter: **"2 of 3 free questions used"**
- After login: small confirmation **"Progress saved"** (one line, auto-dismiss)
- No full dashboard in MVP

5.6 UI / UX requirements (minimal)

| Rule       | Requirement                                                                              |
|------------|------------------------------------------------------------------------------------------|
| Text       | Problem prompt + feedback only; no long marketing copy                                   |
| Buttons    | Max <b>3</b> primary actions on problem screen: Run, Submit, Next                        |
| Navigation | <b>≤ 2</b> top-level destinations: Practice (default), Problems list                     |
| Login      | <b>Hidden during Q1–Q3</b> ; shown <b>only at Q4 gate</b> (+ minimal login/signup pages) |
| Footer     | Optional one line; no multi-column sitemap                                               |
| i18n       | English only                                                                             |
| Mobile     | Responsive problem view (read + submit); code editor best-effort                         |

Remove from UI (hide routes):

- `/pricing``, `/contact`` (or stub)
- `/login``, `/signup`` (linked from Q4 gate only — not in main nav during free tier)
- `/dashboard``, `/create-course``, `/my-courses``
- `/instructor/*``, `/admin/*``
- `/upgrade``, achievements, notifications
- Marketing sections: testimonials, live classes, tour, platform chat bubble

6. Technical Requirements

6.1 Frontend changes

| Task | Description                                                                                                                        | Priority |
|------|------------------------------------------------------------------------------------------------------------------------------------|----------|
| F-01 | New route <code>/`</code> → problem practice (auto-load first/next problem)                                                        | P0       |
| F-02 | New route <code>/problems`</code> → minimal filterable list                                                                        | P1       |
| F-03 | New <code>ProblemView`</code> component (merge patterns from <code>ExerciseView`</code> + <code>PaginatedSkillAssessment`</code> ) | P0       |
| F-04 | Guest submit for Q1–Q3 without auth; auth required from Q4                                                                         | P0       |
| F-05 | Strip <code>Navbar`</code> to logo + optional list link                                                                            | P0       |
| F-06 | <code>bina-practice-guest`</code> <code>localStorage</code> service (read/write/update)                                            | P0       |
| F-07 | Login gate modal at Q4 ( <code>completedCount &gt;= 3`</code> )                                                                    | P0       |

|      |                                                                      |    |
|------|----------------------------------------------------------------------|----|
| F-08 | Post-login sync: call `POST /practice/sync`, then clear localStorage | P0 |
| F-09 | Delete or feature-flag unused route groups from nav                  | P0 |
| F-10 | English-only: stop rendering language selector in MVP shell          | P1 |

## 6.2 Backend changes

| Task | Description                                                                               | Priority |
|------|-------------------------------------------------------------------------------------------|----------|
| B-01 | New module `problems/` — model, routes, service                                           | P0       |
| B-02 | Seed script: 15–25 AI engineering problems                                                | P0       |
| B-03 | `POST /problems/:slug/submit` — guest allowed if `completedCount < 3`; JWT required after | P0       |
| B-04 | `POST /practice/sync` — merge localStorage payload to user account (idempotent)           | P0       |
| B-05 | `UserPracticeProgress` + `ProblemSubmission` models                                       | P0       |
| B-06 | Wire grading: MCQ + AI for open/code (reuse existing services)                            | P0       |
| B-07 | Remove tier quota checks from practice endpoints                                          | P0       |
| B-08 | Reuse existing `auth/` signup + login; add sync hook after successful login               | P0       |
| B-09 | Optional: sync grading (no BullMQ) for faster MVP feedback                                | P1       |
| B-10 | Deprecate/disable unused route mounts in `app.ts` (document only)                         | P1       |

## 6.3 Data model (new)

### Problem

- └─ slug, title, topic, difficulty, type, prompt
- └─ options?, correctAnswer?, starterCode?, rubric?
- └─ active, order, createdAt

### ProblemSubmission

- └─ userId? (null for guest-only local submissions until sync)
- └─ guestSessionId? (set for Q1–Q3 before sync)
- └─ clientSubmissionId (uuid from localStorage – idempotency key)
- └─ problemSlug, answer, code?
- └─ score, correct, feedback, topic
- └─ submittedAt, syncedAt?

### UserPracticeProgress

- └─ userId
- └─ completedSlugs[]
- └─ skillByTopic: { attempted, passed, avgScore, level }
- └─ totalCompleted, lastProblemSlug

```
└─ updatedAt

LocalStorage (client only – see §5.5.2)
└─ bina-practice-guest { submissions[], skillByTopic, completedCount, synced }
```

7. What to Remove / Hide (Phase 1)

Do **not** delete backend code immediately — **hide UI routes** and **stop mounting marketing links**. Backend cleanup in Phase 2.

| Subsystem                      | Action                                                      |
|--------------------------------|-------------------------------------------------------------|
| Course generation & wizard     | Hide routes; no nav links                                   |
| Instructor portal              | Hide                                                        |
| Admin panel                    | Hide (internal env only)                                    |
| Marketplace `/courses` catalog | Hide or redirect to `/`                                     |
| Subscriptions / upgrade        | Hide                                                        |
| Auth pages                     | <b>**Keep**</b> — used at Q4 gate (minimal login/signup UI) |
| Gamification / achievements    | Hide                                                        |
| Learning path → create course  | Remove CTAs                                                 |
| Platform chat bubble           | Remove                                                      |
| Multi-locale UI                | Hide selector; EN copy only                                 |
| Marketing homepage sections    | Replace `/` with practice                                   |

8. Implementation Phases

Phase 0 — Alignment (1–2 days)

- ☐ Client sign-off on this requirements doc
- ☐ Confirm problem types for MVP (MCQ only vs MCQ + code)
- ☐ Confirm free problem limit (**\*\*3\*\*** confirmed)
- ☐ Agree on brand name for minimal shell

Phase 1 — MVP shell (1 week)

- ☐ Problem model + seed data
- ☐ Public API: list, get, submit (guest Q1–3, authed Q4+)
- ☐ New `/`` problem view + feedback panel
- ☐ localStorage guest progress (``bina-practice-guest``)
- ☐ Login gate at Q4 + sync on login
- ☐ Hide all non-MVP routes from navigation
- ☐ English-only UI

**Exit criteria:** User completes 3 problems without login (data in localStorage), hits login at Q4, logs in, data syncs to DB, continues to Q4.

**Phase 2 — Polish (1 week)**

- ☐ `/problems`` browse + filters
- ☐ Code Run button + AI grade for code problems
- ☐ Richer AI feedback prompts
- ☐ Basic analytics (internal)

**Phase 3 — Later (out of MVP)**

- ☐ Admin problem editor
- ☐ Test-case autograder
- ☐ Paid tiers
- ☐ Community / discussions
- ☐ Mobile app

**9. Non-Functional Requirements**

| Category    | Requirement                                              |
|-------------|----------------------------------------------------------|
| Performance | First problem interactive ≤ 3s on broadband              |
| AI cost     | Cap concurrent grading; reuse existing AI usage recorder |
| Security    | Rate-limit anonymous submit; sanitize code sandbox       |

|               |                                                                   |
|---------------|-------------------------------------------------------------------|
| Privacy       | No PII until login; guest UUID + local submissions only for Q1–Q3 |
| Availability  | Single-region deploy acceptable                                   |
| Accessibility | Basic keyboard submit; readable contrast                          |

## 10. Success Metrics (MVP)

| Metric                       | Target (first 30 days after launch)                                    |
|------------------------------|------------------------------------------------------------------------|
| Time to first submit         | Median $\leq$ 60 seconds from landing                                  |
| Problems per session         | $\geq$ 2 average (guest); $\geq$ 4 for logged-in users who passed gate |
| Login conversion at Q4       | $\geq$ 40% of users who complete 3 problems                            |
| Submit completion rate       | $\geq$ 70% of started problems                                         |
| Return visit (same guest ID) | $\geq$ 20% within 7 days                                               |

## 11. Open Questions for Client

1. **Problem types:** MCQ only first, or include coding from day one?
2. **Content source:** Who writes the initial 15–25 problems?
3. **Branding:** Keep "LabPath" or new name?
4. **Free limit:** Confirm **3 questions** before login (or different number)?
5. **Gate timing:** Login before opening Q4, or before submitting Q4?
6. **Signup vs login:** Sign up allowed at gate, or login only?
7. **Skill display:** Numeric score or Beginner / Intermediate / Advanced labels?
8. **Sync conflicts:** If user already has DB progress, merge or overwrite with local?

## 12. Short Checklist (Team Action Items)

### Build

- ☐ `Problem` model + seed bank

- ☐ Guest submit (Q1–3) + authed submit (Q4+)
- ☐ `POST /practice/sync` on login
- ☐ Single-screen practice UI at `/`
- ☐ localStorage guest bundle (`bina-practice-guest`)
- ☐ Login gate at 4th question
- ☐ Minimal chrome (no pricing, no course nav)

## Reuse

- ☐ `exercises/grading.service.ts`
- ☐ `assessments/grading.service.ts`
- ☐ `CodeEditorLab` + code sandbox
- ☐ Existing `auth/` signup + login flows
- ☐ `guestSessionId` pattern from skill assessments
- ☐ `AssessmentSiteShell` (simplified)

## Remove / hide

- ☐ Courses, instructor, admin, marketplace, subscriptions UI
- ☐ Auth on every submit (only gate at Q4)
- ☐ Heavy marketing homepage
- ☐ i18n selector (MVP)
- ☐ Learning path & course creation CTAs

---

## 13. Appendix: Key File References

### Backend

- `b2c-backend/src/modules/assessments/skillAssessment.service.ts`
- `b2c-backend/src/modules/exercises/grading.service.ts`
- `b2c-backend/src/modules/assessments/grading.service.ts`
- `b2c-backend/src/modules/ai-guidance/ai.client.ts`
- `b2c-backend/src/common/constants/aiCategories.ts`



- `b2c-backend/src/app.ts`

## Frontend

- `b2c-frontend/src/features/skill-assessment/`
  - `b2c-frontend/src/features/exercises/components/ExerciseView.tsx`
  - `b2c-frontend/src/features/labs/components/CodeEditorLab.tsx`
  - `b2c-frontend/src/components/marketing/AssessmentSiteShell.tsx`
  - `b2c-frontend/app/page.tsx`
- 

\*End of document\*